LoomMode 2.0 说明书

产品版本 2.0.0 · 开发者 争流儿

第一章 需求与定位

一个人，如何借助 AI 开发复杂的软件工程项目?

AI 编程拓展了个人的开发能力，复杂项目仍然需要持续的专业判断与组织工作。项目逐渐复杂后，需求需要澄清，方案需要比较，不同模块需要协调，修改需要验证，先前形成的决定还要在后续迭代中得到延续。用户面对的挑战，从完成一段代码，扩展到了怎样持续把项目做好。

LoomMode 让个人能够以甲方的身份，委托一支 AI 研发团队开展软件开发。 它是一套面向软件研发的多 AI 角色协作工作流，将规划、约束、审阅、决策与执行组织起来，再通过程序维护任务、权限、状态和证据。用户提出目标、明确重要要求并保留关键决定，团队在授权范围内承担具体研发工作，形成能够检查和继续使用的成果。

LoomMode 把 AI 的专业判断、任务交接和结果检查组织到同一工作流中。直接使用 AI 编程工具时，用户往往还要安排下一步、向不同会话解释背景、转交意见并核对结果。LoomMode 将这些环节纳入工作流，让专业角色负责判断，让程序衔接正式材料与执行事实，并与项目已有的 Git、测试及开发工具配合。

同一套协作机制，既帮助缺少经验的人获得专业支持，也帮助开发者减少重复协调。 当项目需要多轮迭代、跨模块修改、保护已有行为和追溯重要决定时，明确分工与正式交接更有价值。用户获得的是代码改动、验证结果及相应解释，也能够知道哪些问题尚需处理。对于边界简单的一次性小改，流程深度应与实际需要相称。



目前，LoomMode 2.0.0 首发仅支持 Windows 上通过 Codex 运行的受管任务。用户启动并逐次推进流程，任务接受后停在提交授权节点。使用者需要准备项目、较明确的开发目标和必要技术依据；从初步想法研究需求、形成方案的完整前期过程，是后续重点发展方向。

第二章 设计理念与协作模式

2.1 用户委托目标，团队承担专业工作

Owner 提出目标并保留最终授权，AI 团队组织实施。 人类用户作为项目发起人，即 Owner，说明要实现的行为、重要边界和验收期望。AI 团队负责理解要求、规划任务、提出方案、实施与检查。这里的“团队”表达系统内的专业分工与过程责任，实际产品方向、重要风险接受和发布决定仍由人掌握。

AI 团队应在已有授权内作出专业判断，并把真正重要的取舍解释给用户。 日常问题由团队协调和修复；涉及目标变化、重要技术取舍或额外权限时，决策者综合实际证据，说明影响和可选处理方式。决策者本身也是 AI 角色，是统一的人机沟通入口，其判断不能代替必须由 Owner 作出的授权。

2.2 三层递进的产品理念

第一层：让个人具备团队研发能力。 将需求理解、任务规划、执行要求、独立质疑和代码实施组织成共同工作，个人就有机会借助不同专业视角推进项目。角色能力需要通过具体判断和交付体现，共同目标则使这些判断能够落实到同一项成果。

第二层：让团队可靠、自主地工作。 可靠性需要明确目标、合适检查和真实证据；自主性需要明确授权、角色承担日常责任以及程序化衔接。工程机制为团队提供可执行的边界和可检查的依据，使专业判断能够在受控条件下落实。质量与证据可信是减少人工干预的基础。

第三层：让研发团队更加开放、灵活、可演进。 角色经验、模型配置与选择、不同委托范围已经具有机制基础，第三方编程与只读角色调用也开展过实际探索。后续可以据此增强专业能力、调整协作方式，并逐步覆盖更完整的研发过程，使工作流适应不同用户和项目。

2.3 五个角色，共同对目标负责

多角色解决专业分工问题，工程机制解决协作过程的控制与交付问题。 LoomMode 固定设置五个角色，分别承担不同判断责任；同一模型体系也可以通过不同调用和上下文承载这些角色。角色价值来自有依据的意见、明确的处理责任和实际执行，专业正确性仍需通过检查与成果判断。

角色	专业责任	与其他角色的主要关系
决策者	综合重要取舍、处理升级问题，向用户解释状态与方案	在授权内协调；超出边界或涉及保留决定时提交 Owner
推进者	选择任务、组织推进、判断任务成果是否可以接受	协调普通任务分歧，推动决定落实
约束者	将任务意图转化为正式执行要求	判断讨论意见的采纳方式，形成最终约束
讨论者	独立检查意图、方案与约束之间的偏差	提出有依据的建议，由相应责任角色处理
执行者	核对现场、实现、测试、自检、修复并报告	依据正式输入工作，交回改动与执行结果

普通分歧由团队处理，重要分歧按责任升级。 讨论者提出建议，约束者作采纳判断；普通任务中的实质分歧由推进者协调，重大取舍交决策者综合处理。任何角色的专业意见都不能自行扩大授权。独立代码审查是按需安排的活动，普通任务也无需五个角色每次依次发言。

2.4 从真实项目中形成协作方法

SciLoom 科研舱和 SciLoom 电子蛐蛐的真实开发，为 LoomMode 提供了方法来源和实践检验。 开发者争流儿起初缺少软件开发经验，在借助 AI 开展这些项目时，逐步形成多角色分工、正式任务约束、专业审阅、实施与结果检查的方法。这些方法在实际开发中被应用、修正，也暴露了人工搬运资料、切换会话和安排返修的组织负担。

LoomMode 将实践中的协作方法逐步工程化，再通过程序运行继续检验。 团队从实际开发需求出发，将多角色协作方法写入工作流，并通过实现、修复与验证逐步形成 LoomMode 2.0.0。第 3.6 节以电子蛐蛐的真人参与任务展示协作方法如何用于实际开发。第 4.4 节列出程序化运行和 2.0.0 发布前验证的对应记录。

第三章 核心功能与使用过程

3.1 准备项目与委托材料

首次使用需要同时准备运行环境、项目现场和清楚的任务目标。 当前采用本地命令行与脚本入口，基本环境包括 Windows、Python、Git 和可正常使用的 Codex。源码声明 Python 3.11 及以上，具体工具版本与组合以当前使用文档为准。完整使用还需要配套工具、配置、角色材料和必要测试条件，安装一个 Python 包并不等于完成全部准备。

用户提供已有项目或相关资料，说明希望改变的行为、需要保护的要求、允许操作的范围和完成标准。可以先用自然语言表达这些内容，再由团队整理成执行依据。对技术要求尚不熟悉的用户，决策者可以在讨论中帮助解释取舍；只有初步产品创意时，仍需先准备足以进入开发的需求与技术依据。

委托输入示例（说明性）

> 我希望为现有软件增加 CSV 导出功能。导出应使用已有记录，并保留原有的数据保存方式。请检查导出内容是否与源记录一致，并验证原有保存功能。

团队可据此把实施范围收敛到导出入口和数据读取，将原有保存行为列为保护要求，并用导出内容核对及保存功能回归作为验收条件。执行前还需把涉及的资源与 Owner 提供的授权逐项对应。

3.2 启动受管任务，按节点开展协作

首次使用的主要过程是：准备项目与目标，启动受管任务，逐次推进，检查交付结果，再处理适用的授权停点。 用户完成配置后，通过当前入口启动任务并发起后续推进，每次等待本轮活动结束再继续。程序根据已保存节点组织角色、装配指定材料并保存结果；用户无需为每次角色交接重新编排全部正式输入。

人工推进是一项运行操作；是否接受方案、是否增加权限，则是另一类专业或授权决定。团队在每次活动中负责相应判断，程序负责调度和核验，Owner 在需要时作关键选择。这样可以看清当前仍需哪些人工动作，也能看清哪些组织工作已经由系统承担。

规划阶段，推进者选择一项连贯、可验收的工作；约束者明确范围、输入、产物和检查要求；讨论者指出遗漏或冲突；约束者再形成正式版本。必要的审阅关注原始目标是否得到落实，紧密依赖的实现、接入和局部修复可以在同一目标下连续完成。

3.3 开发、测试与问题处理

执行阶段交付的不只是代码，还包括与本次修改相对应的检查结果。 执行者先核对工作区、已有修改、正式输入和工具条件，再实施开发，运行与目标相符的测试并自检。涉及需要固定验证的任务，还要保留规定检查的实际执行依据。

原因明确、直接相关且在授权内的小问题，可以由执行者修复并复验。方案或范围需要调整时，由相应角色判断；只有报告格式或叙述存在问题时，可以补正报告而保留原实现及其证据。模型调用、环境、权限和业务代码失败分别处理，避免把不同问题都归为“再试一次”。

遇到效果不明的中断，系统停止受影响动作，保留已知调用、修改和资源状态，形成问题说明。决策者再根据事实协调下一步，涉及额外权限或无法可靠处置的事项交回 Owner。停止后的工作区可能已有改动，继续前需要核对现场，安全停止也不代表自动撤销修改。

3.4 检查交付，理解任务终点

任务结束后，用户得到项目中的实际改动、执行报告、验证依据和待处理事项。 推进者结合目标与证据判断是否接受，用户则可以检查变更说明、运行项目对应功能，确认结果是否符合自己的使用预期。团队报告应解释完成了什么、还有什么限制，以及哪些事项需要继续决定。

首发路径在任务接受后的提交授权节点停下，不自动执行 Git 提交。用通俗的话说，代码和相关记录已经形成，但还没有自动写入版本库的一次正式提交；后续提交、推送、打包和发布，需按各自要求处理。用户应核对差异、保留成果，再决定下一项工作。

3.5 适用场景与使用投入

更适合引入完整协作的，是需要保护既有行为、反复迭代和追溯判断的任务。 多角色调用、串行等待、正式材料维护和异常核对都会增加投入，应与任务的复杂度和风险相称。开源许可也不包含外部模型服务，调用费用取决于实际账户和接入方式。当前以机制作用和已有工程记录说明价值，尚不据此承诺统一的耗时或费用节省比例。

任务、任务组和阶段等不同委托范围已有机制表达，更大范围仍保留相应审查和结算责任。当前首发从单任务入手，后续根据真实运行经验完善连续托管，让用户在合理的准备与管理成本下逐步扩大委托。

3.6 真实开发实践 让用户从旁观者变成游戏参与者

在 SciLoom 电子蛐蛐的 AC1-HUMAN 任务中，Owner 希望让真实登录用户拥有自己的游戏席位，与 AI 人物共同参与。 电子蛐蛐已有多个围绕 AI 虚拟人物（AI Character）互动的场景，真人参与方式各不相同。这次开发需要完善共同参与的流程，同时保护原有界面和操作习惯、各场景的实际规则、真人与 AI 的操作权限，以及隐藏信息和游戏人物经历的连续性。

一个参与入口牵动了多处相互依赖的改动。界面需要呈现真人可用的操作，后端要识别行动者和席位，游戏规则与状态需要支持真人和 AI 共同参与，游戏结束后还要保存 AI 人物的经历。用户坐到自己的席位后，应当能够按规则行动，也只能看到该席位有权获取的信息。

Owner 确定目标和边界，Codex 主代理组织实施、集成、测试和报告。后端代理（human_backend）处理真人席位、权限与经历保存，界面代理（human_ui）实现真人操作与交互，场景审查代理

（scene_audit）开展独立检查、测试和复验。该任务采用 Owner 授权的 Codex 多代理直接开发方式，是 LoomMode 协作方法的真实实践来源；Engine 的受管运行验证另见第 4.4 节。

团队在既有系统上完善了 10 个游戏场景的真人参与能力，并保留人物茶话会原有的用户交互。 任务涉及的 11 个正式场景中，物资合作已有真人席位，本轮扩展其与长期 AI 人物共同参与及经历保存的路径；其余 9 个游戏接入真人参与流程。完善后的游戏支持登录用户进入席位、完成合法操作并与 AI 一起推进至游戏结束，同时维护行动权限与隐藏信息隔离。对用户而言，变化是可以从观看 AI 互动，转向在同一套游戏规则中与它们共同参与。

实际运行也暴露了仅检查代码不容易发现的问题。浏览器验证发现，合作猜词在用户提交后出现渲染崩溃；团队修复了相关数据处理，并补充逐步渲染检查。独立视觉审查则发现，斗地主的手机手牌选择区纵向过长，影响牌局界面的使用；团队局部调整布局，再进行整局运行和多屏复验。两处问题都完成了针对性修复及后续检查。

验证结合自动化测试、实际页面操作和数据库检查开展。浏览器部分使用 Chromium、实际登录认证、合成账户和隔离 PostgreSQL 数据库，由脚本操作真人席位，并通过可控的模型替代服务（Stub）提供 AI 响应。记录覆盖正常游戏完成、预期 AI 经历保存，以及已保存经历进入代表性后续茶话会请求的路径，使游戏结束后的经历能够继续作为互动输入。

必要产品回归按各测试文件最后一轮完整结果汇总，最终台账记录 **1050 项通过、4 项条件跳过。**独立审查代理还完成了 **59 项聚焦测试和 148 项专项补验，均通过。**这两组测试均包含在上述 1050 项覆盖内，彼此也有重叠，分别体现复验工作，不另行累加。该次交付记录确认开发验证已完成，交付时尚无 Owner 人工接受记录。

这次实践展示了多 AI 协作的工程价值：围绕同一个用户目标，将专业分工、跨模块集成、问题修复和独立检查组织成实际交付。 后端、界面与审查各自承担专业工作，主代理负责将这些工作衔接起来。团队既实现了新的参与方式，也通过运行和复验检查它与已有系统是否配合。

第四章 技术架构与实践验证

4.1 架构 将专业判断、流程控制与运行事实分开

LoomMode 的工程架构围绕一条责任链展开：人确定委托，AI 形成专业判断，程序核验并组织执行，再把结果交回判断。 核心工作流以 Python 实现，首发通过 Codex 的程序化接口接入模型与开发能力。以下组件分别掌握不同信息，共同组织一次受管任务。

AI 角色运行层读取明确绑定的角色定义，组合当前活动要求、适用材料和工作状态，将模型返回解释为专业结论。工作流引擎（Workflow Engine）维护节点和合法动作，检查返回角色、材料及产物是否符合当前流程，再保存结果、转交后续活动。引擎处理可确定的规则，任务价值、方案优劣和业务结果仍由相应角色判断。

运行宿主（Host）连接程序规则与真实环境，负责工作区身份、运行装配、权限与资源检查，以及执行和测试事实的收集。执行者（Executor）承担实施责任，通过配置的受管后端调用开发工具。执行者是专业角色，执行后端是实际工具通路，两者配合使任务要求落实为文件修改、命令执行和检查结果。

状态存储保存项目、任务、节点、正在进行的调用、修复和阻断等运行信息；正式资料存储保存约束、报告及其版本和引用。前者回答流程走到哪里，后者回答依据和成果是什么。运行状态与专业文稿分别维护，有助于避免把一句“已经完成”误当作实际执行终态。

在正常流转中，引擎依据当前节点请求相应角色工作，角色运行层准备输入，宿主与后端在实际调用边界执行核验。返回的专业结论、执行事实与正式产物再交由引擎关联和保存，符合条件后才能进入下一步。由此，模型的意见、程序的核验和环境中发生的动作各有来源。

4.2 七项相互配合的工程机制

4.2.1 委托合同与任务约束 把目标落实为共同依据

自然语言目标需要转化为可实施、可检查的要求。推进者提出任务意图，约束者将目标、范围、必要输入、预期产物、验收和停止条件组织为正式任务约束（Task Constraint），经审阅和定稿后交给执行者。它使开发与结果检查围绕同一份要求展开。

任务约束说明应该做什么，真实授权决定允许做什么。 程序将授权来源与项目、工作区、委托成员和适用资源关联；生成一份约束或建议不能自行扩大权限。这样既保留 AI 对实施细节的专业空间，也使范围变更有明确入口。代价是任务开始前必须准备真实、可读且相互一致的依据。

以第 3.1 节的 CSV 委托为例，约束可以要求读取现有记录并生成导出文件。若拟采用的实现还需要更改授权范围外的存储配置，Host 应在执行入口依据实际授权核验。团队需要把方案收回已有授权范围，或由 Owner 明确追加授权后再推进。

4.2.2 多角色协作与审阅 让意见有处理责任

同一个方案需要被提出、质疑和落实。普通流程把任务选择、约束编制、审阅建议、约束定稿、实施及结果判断分开，角色之间传递明确结论和必要理由。约束者负责判断建议怎样采纳，推进者负责协调普通任务和判断结果，重要分歧再进入决策流程。

角色运行层按活动读取指定材料和职责，独立代码审查按需要使用新鲜、隔离的上下文。审阅发现的问题应能回到原始目标和证据，形成具体处理；更换角色名称或模型本身不足以证明独立性。这种分工为发现偏差提供机会，也增加调用与交接成本，因此审阅深度需要与任务相称。

4.2.3 状态机与流程 Gate 约束何时能够继续

自动化需要区分“准备执行”“已经执行”和“结果可接受”。引擎通过状态机规定节点、负责角色和允许动作，检查返回结构与正式产物，保存成功后再推进。执行尚在进行、返回不合法或必要资料未形成时，流程不能按正常完成继续。

Gate 即进入后续步骤所需满足的关口条件。有些检查材料和结果是否齐备，有些保留人类授权。状态持久化同时记录正在进行的活动和已发生的处理，减少重启时误判工作位置的风险。状态机能检查合法转换与必要机械前提，方案是否合理、测试是否充分仍需要专业判断。

4.2.4 正式材料与上下文保真 保证读到的是本次依据

长期协作容易出现版本错配、摘要失真或只收到引用却读不到正文的问题。LoomMode 为正式产物保留明确版本，结合内容摘要、任务归属和引用校验识别材料；读取时重新核对身份，再向角色提供本轮指定原文和必要事实。内容摘要可以理解为识别文件内容是否变化的校验标记。

正式记录变更通过新版本或补充向前表达，保留原始结论。程序应直接绑定自己掌握的机械事实，角色则解释材料并作专业判断。这样减少了交接中重新抄写身份、凭记忆选取“最新版本”的风险；同时也要求维护引用和原文可读性。材料身份一致，只解决共同依据的问题，内容是否正确还要审阅。

4.2.5 程序级授权与受管执行 让边界在实际入口生效

提示词中的范围要求需要实际执行通路配合。Host 在受管入口核验项目、工作区、授权和资源条件；工具策略区分只读判断、实现、测试等活动，执行后端结合路径限制、工具权限和相应沙箱落实约束。模型建议经过这些检查，才可能成为实际操作。

工作区身份与写入锁用于限制同时活动的受管写入者，宿主还管理自己拥有的执行进程与资源。顺序调度和单写入者降低了多个执行过程相互干扰的复杂度，也牺牲了并行开发的吞吐。保护范围对应已纳入管理的入口，用户另开工具或外部进程的改动仍需核对；现有未提交修改不能被随意归入本次成果或清理掉。

4.2.6 执行证据与结果验收 让报告对应实际工作

模型能够描述测试通过，程序需要保留测试确实执行的事实。固定测试机制记录批准的检查命令、工作位置与时限，宿主将实际结果关联到任务、上下文、执行后端和对应实现依据，再与报告一并交给结果判断。报告与原执行输出建立关联，有助于发现叙述与运行事实不一致。

执行结束、测试通过、任务接受和提交授权分别记录。推进者依据目标、正式报告和宿主事实判断接受；相关代码再次变化时，按影响补充验证，历史结果只在仍适用的范围内复用。**流程受控、执行事**

实可核对和软件满足需求，是相互支撑的三个层次。 证据使判断有据可查，软件质量还取决于需求理解、测试覆盖和审阅质量。

例如，执行报告写着“检查通过”，但本次任务所需的固定测试缺少宿主回执。此时结果判断仍缺一项依据。补齐对应的执行记录或完成适用的验证后，推进者才能结合报告判断任务是否可以接受。

4.2.7 有界修复与安全停止 保留成果，也保留未知

错误需要按原因处理。直接相关、授权内且原因明确的局部问题可以修复；报告补正只纠正原报告，不重新实施或测试。对同一目标的反复尝试保留历史与限额，不能通过换编号、目录或模型把既有失败清零。

例如，执行超时时，代码可能已经写入工作区。宿主分别保留调用是否发出、是否收到最终结果、已知副作用和资源清理情况。若执行效果仍不清楚，系统暂停相关调用，先核对文件改动与运行记录，再决定保留、修复或接续。这样可以避免把已有修改与下一次执行叠在一起。

4.3 机制如何协同 一次任务的责任、核验与产物

同一任务贯穿目标、约束、执行和接受，工程机制在交接处共同生效。 下表概括 LoomMode 受管任务的正常流转。第 4.4 节的受管任务记录提供对应的程序化验证依据。

环节	专业判断与实际执行责任	程序核验及形成的材料	进入下一步的条件
明确委托	Owner 确定目标和授权，团队理解要求	Host 核项目、工作区、授权来源及资源；保留委托依据	输入可读，范围与运行条件适用
组织任务	推进者选择工作，约束者编制要求，讨论者审阅	引擎核当前角色与合法返回，保存意图和审阅材料	分歧按职责处理，正式约束形成
准备执行	执行者核现场和实施条件	Host 核授权、材料身份、工具策略及写入所有权	当前任务具备实际执行资格
开发与测试	执行者经受管后端修改、测试和必要修复	宿主收集执行结果、固定测试回执与实现依据；保存报告	取得已知结果，必要验证与报告齐备
判断接受	推进者对照目标和证据判断	引擎关联报告与原执行事实，核合法结果路由	结果被接受，或进入合法修复 / 决策处理
到达授权停点	Owner 处理适用的人类决定	保留接受记录与实际停点；首发不自动提交 Git	后续动作另按授权及现场条件处理

这些环节共同避免目标、实现和验证各自脱节。程序维护可确定的身份、状态和事实，AI 围绕这些依据作判断，用户在关键边界保留决定权。

4.4 实践来源与已有验证成果

LoomMode 的工程依据来自真实项目、受管任务运行及 2.0.0 发布前验证。 科研舱与电子蛐蛐提供协作实践。受管任务记录说明实现、测试和接受如何衔接。2.0.0 候选版本的构建与测试核对了源码、配置和分发的一致性。

验证对象	阶段与条件	已有结果	证明范围
真实项目中的协作方法 [E1]	科研舱、电子蛐蛐的多角色开发实践；AC1-HUMAN 采用 Owner 授权的 Codex 多代理直接开发	完善 10 个游戏场景的真人参与，保留茶话会原有交互；完成猜词渲染与斗地主手机布局修复，并通过产品回归和独立复验	提供真实开发、专业分工、集成与检查的实践依据，为协作方法逐步程序化提供经验
受管第三方编程与前后任务交接 [E2]	历史 LM2-C3 专项，Windows、Codex CLI 0.162.1 MXC、Python 与 DeepSeek Responses 直连	独立测试工程先完成计分汇总，再完成 CSV 导出。两项任务被接受，并有前后提交关联。分别通过 11 项、18 项固定检查，存在重叠	证明特定后端和任务条件下的工程交付与成果交接，不扩大为首发通用多模型支持
Codex 受管单任务 [E3]	历史 LM2-C5-A12，Codex-only、本地受控现场，2026-10-28 交接	已有原宿主测试、正式报告和任务接受记录，停在提交授权节点，资源收束有记录	证明相应真实任务链，区别于长期多任务及整个阶段收尾验证
模型选择与只读角色调用 [E4]	历史 LM2-C4 多 AI V1 专项，明确配置、读取范围及调用额度	实际完成偏好驱动的模型选择和另上下文审阅，审阅提出补充正负数混合输入断言的建议	证明选择与静态审阅路径。该建议未在这次只读验证中执行
候选构建与安装 [E5]	2.0.0 候选，2026-10-30 定稿记录。实测 Python 3.13.5 工具链	两个独立输出根构建产物逐字节一致，选定源码能够重建对应核心包。新环境隔离安装并核对 65 个模块来源	支持源码、包与安装来源的一致性，区别于真实开发效果和任意设备兼容
候选流程与相关回归 [E5]	2.0.0 候选。合成模型通信和相应测试环境	候选打包及人工停点 16 项通过。相关回归 94 个独立用例最终各有适用通过依据，首轮 93 项通过、1 项失败，修正旧测试夹具后对失败项定向复验通过	验证相应控制、角色加载和分发路径。该轮没有新增真实模型调用

证据索引

E1 科研舱与电子蛐蛐的实践来源见项目定位材料；AC1-HUMAN 案例依据已接受的 LoomMode_AC1-HUMAN_案例事实核对报告_v1.0 第 2—6、9 节，对应执行报告 AC1-HUMAN_EXECUTION_REPORT_20261102_v001、独立审查记录 AC1-HUMAN_INDEPENDENT_REVIEW_20261102_v002 与 AC1-HUMAN_INDEPENDENT_REVIEW_20261102_v003。场景范围、问题修复、测试口径及交付状态见第 3.6 节。

E2 LM2-C3_P6_Whole_Goal_Fresh_Independent_Acceptance_Report_v001_20261025 第 2.3 节及第 4.3 节 E3，配套接受记录 LM2-C3-A40_ACCEPTANCE_v001、LM2-C3-A41_ACCEPTANCE_v001。

E3 LM2-C5_Managed_Continuation_Concentrated_Handoff_v002_20261028，配套单任务接受记录 SMK12-A1-A1_ACCEPTANCE_v001。

E4 LM2-C4_Multi_AI_V1_Unified_Owner_Acceptance_Report_v001_20261025，参见真实选择、审阅意见和 API 运行记录。

E5 LM2-D1-B3_Execution_Report_v001_20261030 第 2、4、5、7、8 节，涵盖候选身份、构建与安装、测试与旧失败处理，以及交付停点。

历史专项按任务和阶段标识追溯，产品版本以各自记录为准。测试结果按各轮条件和覆盖范围分别列示。2.0.0 首次公开支持聚焦 Windows、Codex-only、受管单任务，长期托管和更广的适配继续通过真实项目完善。

第五章 开源定位与未来发展

LoomMode 由争流儿独立开发，采用 Apache License 2.0，以 GitHub 作为代码、文档和版本信息的主要对外入口。开放项目希望让更多人理解、使用和改进这套协作方法，并以真实问题推动后续演进。

角色可演进，让项目经验能够服务后续工作。 现有机制支持将问题反馈、阶段知识和角色经验分层整理，保留来源与适用条件。角色可以据此获得更合适的工作依据，固定职责和权限继续受控维护，使经验复用服务于专业判断。

能力可扩展，让成熟工具补充团队的专业能力。 后续希望吸收 Skills、测试工具及专业工作流，在研究方案、编码、检查和文档等方面增强现有角色。整合重点是能力是否适用、输入输出能否衔接、结果能否验证；已有协作职责为这些能力提供共同的组织位置。

模型可选择，让同一协作框架容纳不同能力来源。 模型目录、角色路由和偏好选择已有实现，第三方编程与只读 API 路线也取得了限定验证。后续需要继续完善兼容与实际使用，使用户能根据任务选择能力，同时保持授权和证据口径一致。模型可配置与各模型获得相同执行资格，是不同问题。

协作可调整，让自主程度与任务风险及用户需要相称。 不同委托范围、阶段治理和必要的人类关口已有机制基础，首发先提供受管单任务路径。后续通过运行反馈完善连续托管，兼顾愿意深入参与的开发者和希望减少日常协调的用户。

研发过程可延伸，让团队进一步帮助用户把想法变成开发依据。 当前重点在技术依据较明确后的实施阶段，未来希望向需求澄清、可行性研究、方案比较和技术方案形成延伸。对只有初步想法的人，这将进一步降低进入软件研发的组织门槛。

项目参与可以从使用反馈、问题复现、文档、测试、适配及角色能力建设等具体工作开始。
LoomMode 希望通过可检查的任务成果和持续积累的实践，帮助个人逐步组织起能够承担更多研发工作的 AI 团队。